Universidad Tecmilenio

Sistemas y tecnologías de administración del conocimiento

MAPS - Avance de Reto/Proyecto

Prototipo de Sistema de Gestión del Conocimiento para NexaSoft Consulting

Alumno: Vicente Alberto Jahrift Alonso López

Matrícula: AL07002967

Carrera: Ingeniería en Desarrollo de Software

Fecha: 28 de octubre de 2026

1. Descripción de la empresa ficticia

NexaSoft Consulting es una empresa ficticia de consultoría de TI con sede en Monterrey. La inventé tomando como modelo a las consultoras reales con las que trabajé en actividades anteriores, como Neoris. Tiene unos 300 consultores y trabaja con clientes de manufactura, retail y banca.

a. Tipo de organización

Es una consultora de tecnología. No vende un producto propio: hace y mantiene sistemas para otras empresas.

b. Actividades principales

- **Desarrollo de software:** apps web y móviles, integraciones y modernización de sistemas viejos.
- **Soporte técnico:** mesa de ayuda para los sistemas que entrega, incluyendo SAP.
- **Infraestructura y nube:** servidores, respaldos, monitoreo y seguridad.
- **Gestión de proyectos:** planeación, tiempos, costos y comunicación con los clientes.
- **Recursos humanos y cumplimiento:** políticas internas, capacitación e incorporación de empleados.

c. Tipos de conocimiento y documentos que maneja

Tipo de documento	Qué contiene	Área que lo genera	Clave
Solución de ticket y guía de mesa de ayuda	Problema, causa y pasos para resolverlo	Soporte técnico	SOP
Documentación técnica y estándares	APIs, manuales de despliegue, reglas de código	Desarrollo	DEV
Reporte de incidente y procedimiento	Qué falló, cuánto duró, cómo evitarlo	Infraestructura y nube	INF
Lección aprendida y reporte de avance	Qué funcionó, riesgos, estimaciones	Gestión de proyectos	PRY
Política interna	Reglas de seguridad, trabajo remoto, ética	Recursos humanos	POL

¿Por qué necesita un sistema de gestión del conocimiento?

En NexaSoft cada área genera muchos documentos, pero están en carpetas distintas y cada quien los guarda a su manera. Los consultores cambian de proyecto seguido, así que cuando se van, se llevan lo que saben. Esto causa tres problemas:

- Se resuelven los mismos tickets una y otra vez porque nadie encuentra la solución anterior.
- Los documentos nuevos no se clasifican, entonces el repositorio se vuelve un desorden.
- Buscar por palabra exacta no sirve, porque cada persona escribe el problema diferente.

Por eso propongo un sistema que haga dos cosas: clasificar los documentos solos en su categoría y buscarlos con preguntas normales. Las cinco categorías del clasificador son las mismas cinco áreas que generan los documentos.

Áreas que generan, usan y actualizan la información

Área	Genera	Usa	Actualiza
Soporte técnico	Soluciones de tickets	Soluciones, documentación técnica	Soluciones que ya no sirven
Desarrollo	Documentación técnica, estándares	Soluciones de tickets, lecciones	Manuales cuando cambia el sistema
Infraestructura y nube	Reportes de incidentes, procedimientos	Procedimientos, documentación técnica	Procedimientos después de cada incidente
Gestión de proyectos	Lecciones aprendidas, reportes de avance	Lecciones y plantillas de estimación	Plantillas con datos de proyectos nuevos
Recursos humanos	Políticas internas	Políticas	Políticas cuando cambian las reglas o la ley

2. Mapa del ciclo de vida del conocimiento

El mapa muestra las seis etapas que pide la actividad. En el centro está el repositorio, que es donde viven todos los documentos. También marqué en qué etapas entra el prototipo.

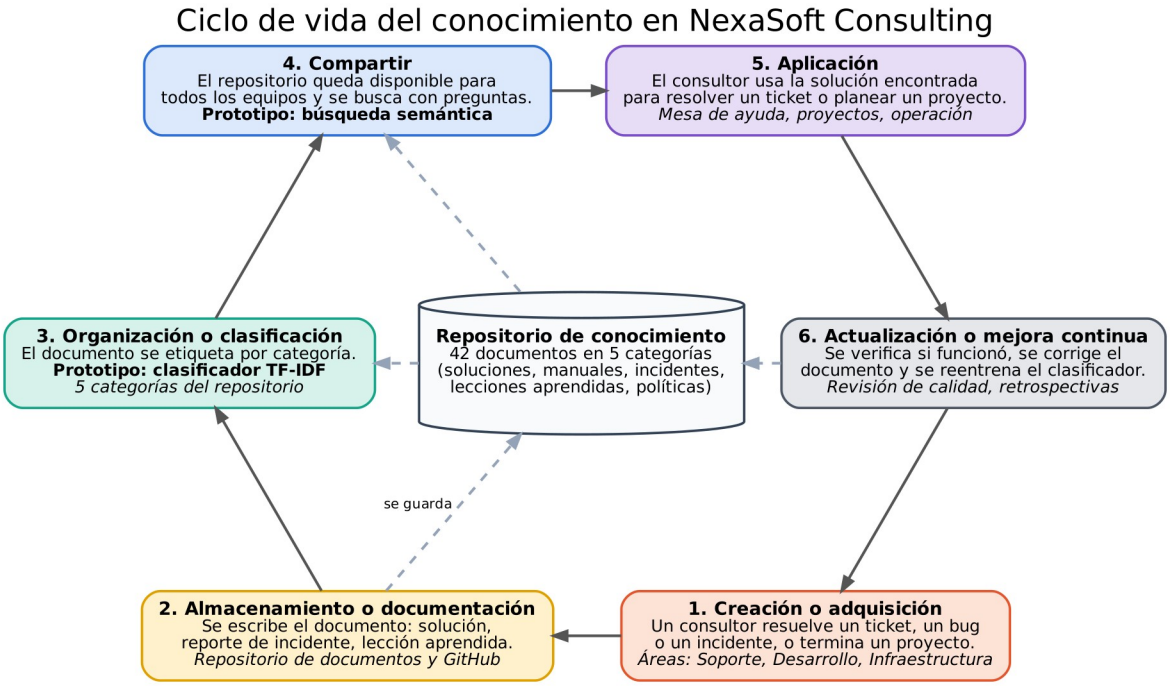


Figura 1. Ciclo de vida del conocimiento en NexaSoft Consulting (hecho con Graphviz).

Etapa	Qué pasa en NexaSoft	Ejemplo
1. Creación o adquisición	Alguien resuelve un problema o termina un proyecto y aprende algo.	Un agente desbloquea una cuenta de usuario.
2. Almacenamiento o documentación	Lo que aprendió se escribe en un documento y se sube al repositorio.	Se escribe SOP-008 con los pasos.
3. Organización o clasificación	El documento se etiqueta en una de las cinco categorías. Esto lo hace el clasificador.	SOP-008 queda en Soporte técnico.
4. Compartir	Todos los equipos pueden buscar en el repositorio con preguntas normales. Esto lo hace la búsqueda semántica.	Otro agente busca "cuenta bloqueada".
5. Aplicación	La persona usa la solución en su trabajo.	El agente aplica los pasos y cierra su ticket.
6. Actualización o mejora continua	Se revisa si funcionó. Si no, se corrige el documento y se vuelve a entrenar el clasificador.	Se agrega un paso nuevo a SOP-008.

3. Ontología del conocimiento

La ontología muestra cómo se conecta el conocimiento dentro de NexaSoft. Usé tres tipos de elementos: conceptos del dominio, tipos de documento y áreas. Las flechas son las relaciones.

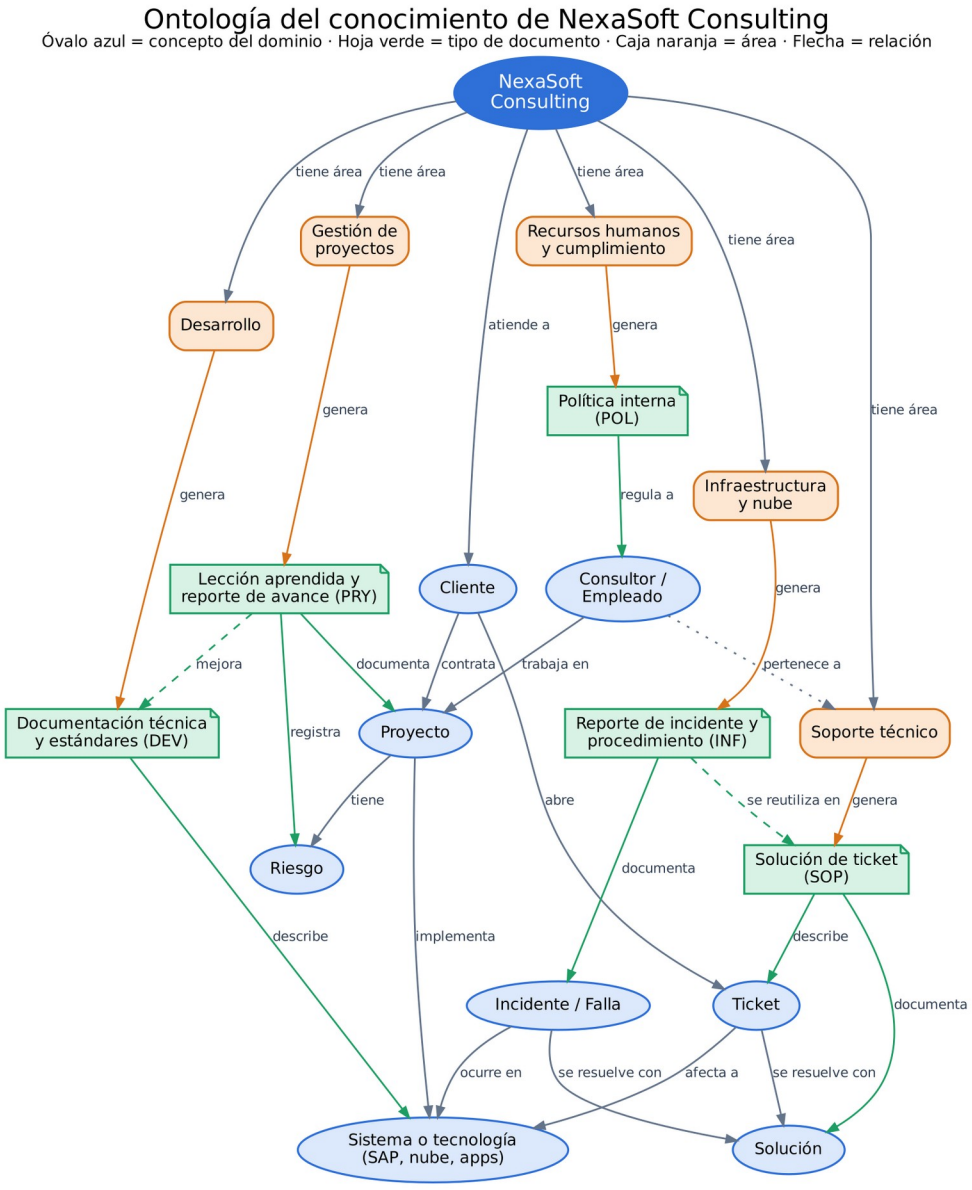


Figura 2. Ontología o grafo conceptual del conocimiento de NexaSoft Consulting.

a. Conceptos principales del dominio

NexaSoft Consulting, Cliente, Proyecto, Consultor o Empleado, Sistema o tecnología, Ticket, Incidente o falla, Solución y Riesgo.

b. Tipos de documento

Solución de ticket (SOP), Documentación técnica (DEV), Reporte de incidente (INF), Lección aprendida (PRY) y Política interna (POL).

c. Relaciones entre conceptos

Concepto	Relación	Concepto
NexaSoft	atiende a	Cliente
Cliente	contrata	Proyecto
Consultor	trabaja en	Proyecto
Proyecto	implementa	Sistema o tecnología
Cliente	abre	Ticket
Ticket	afecta a	Sistema
Ticket / Incidente	se resuelve con	Solución
Proyecto	tiene	Riesgo

d. Relaciones entre documentos y conceptos

Documento	Relación	Concepto u otro documento
Solución de ticket	documenta / describe	Solución / Ticket
Documentación técnica	describe	Sistema o tecnología
Reporte de incidente	documenta	Incidente o falla
Lección aprendida	documenta / registra	Proyecto / Riesgo
Política interna	regula a	Consultor o Empleado
Reporte de incidente	se reutiliza en	Solución de ticket
Lección aprendida	mejora	Documentación técnica

La ontología se conecta con el mapa del ciclo de vida. Cada área crea su tipo de documento en la etapa 1. Cada tipo de documento es una de las categorías del clasificador en la etapa 3. Y las relaciones explican por qué la búsqueda encuentra documentos de varias áreas, como un ticket de SAP que también aparece en la documentación técnica de la integración.

4. Explicación del prototipo

El prototipo está hecho en Python con scikit-learn. Tiene tres archivos: utilidades.py carga los documentos, clasificador.py los clasifica y busqueda_semantica.py los busca.

Documentos de prueba

Creé 42 documentos cortos en español, uno por archivo .txt. Cada carpeta es una categoría: soporte técnico (9), desarrollo de software (9), infraestructura y nube (8), gestión de proyectos (8) y políticas internas (8). Aparte hay 5 documentos nuevos sin categoría para probar el clasificador.

Cómo funciona el clasificador

1. **Carga los documentos.** Lee cada .txt y toma el nombre de la carpeta como su categoría.
2. **Separa los datos.** 75% (31 documentos) para entrenar y 25% (11) para probar.
3. **Vectoriza con TF-IDF.** Convierte cada texto en números. TF-IDF le da más peso a las palabras que aparecen mucho en un documento pero poco en los demás. Quité acentos y palabras vacías como "de", "la" y "que".
4. **Entrena dos modelos.** Naive Bayes y Regresión Logística. Los comparo y me quedo con el que tiene mejor exactitud.
5. **Clasifica documentos nuevos.** Le paso los 5 documentos nuevos y me dice su categoría y qué tan seguro está.
6. **Guarda resultados.** La matriz de confusión y el modelo entrenado quedan en la carpeta resultados.

Cómo funciona la búsqueda semántica

1. **Vectoriza todos los documentos** con TF-IDF y guarda la matriz.
2. **Recibe una consulta** en lenguaje normal, por ejemplo "el usuario no puede entrar porque su cuenta está bloqueada".
3. **Convierte la consulta a vector** con el mismo vocabulario de los documentos.
4. **Calcula la similitud coseno** entre la consulta y cada documento. Va de 0 (nada que ver) a 1 (igual).
5. **Muestra los 3 documentos** con la similitud más alta, con su categoría y su puntaje.

Parte clave del código:

```
vector_consulta = vectorizador.transform([consulta])

similitudes = cosine_similarity(vector_consulta, matriz_documentos)[0]

orden = similitudes.argsort()[::-1][:top]
```

Nota: con TF-IDF la búsqueda se basa en las palabras que comparten la consulta y los documentos. Por eso funciona bien cuando la pregunta usa palabras parecidas. En una versión futura se podrían usar embeddings para entender sinónimos.

5. Evidencias del prototipo

5.1 Clasificador: carga, vectorización y entrenamiento

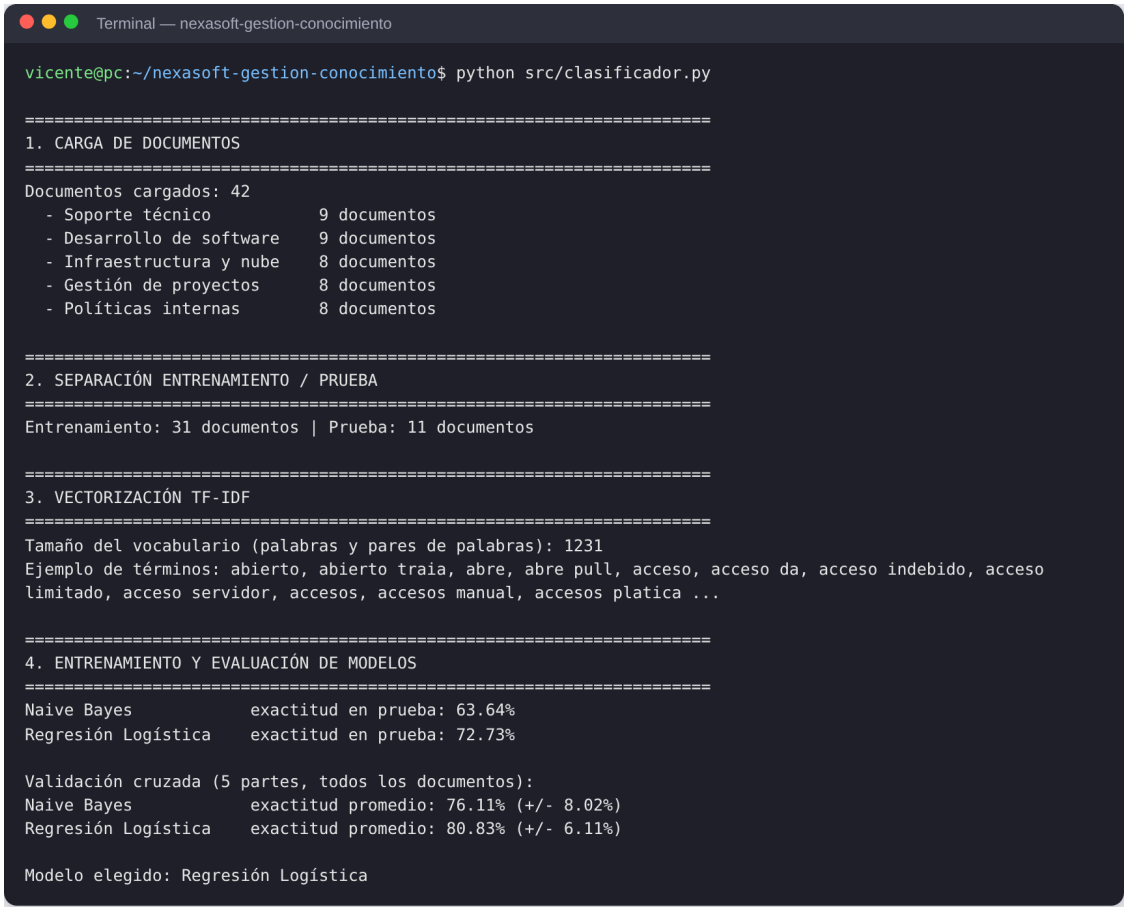


Figura 3. Ejecución de clasificador.py: documentos cargados, TF-IDF y comparación de modelos.

Con la prueba de 11 documentos, Regresión Logística sacó 72.73% y Naive Bayes 63.64%. Como son pocos documentos, también corrí una validación cruzada de 5 partes. Ahí Regresión Logística sacó 80.83% en promedio y Naive Bayes 76.11%. Por eso elegí Regresión Logística.

5.2 Clasificador: resultados de la prueba

```
Terminal — nexasoft-gestion-conocimiento

vicente@pc:~/nexasoft-gestion-conocimiento$ python src/clasificador.py  (continuación)
Resultado por documento de prueba:
ID       Categoría real            Predicción                ¿Correcto?
PRY-008  Gestión de proyectos      Gestión de proyectos      Sí
DEV-008  Desarrollo de software    Desarrollo de software    Sí
POL-002  Políticas internas        Políticas internas        Sí
DEV-009  Desarrollo de software    Soporte técnico           No
INF-008  Infraestructura y nube    Gestión de proyectos      No
PRY-005  Gestión de proyectos      Gestión de proyectos      Sí
INF-004  Infraestructura y nube    Infraestructura y nube    Sí
POL-004  Políticas internas        Políticas internas        Sí
SOP-003  Soporte técnico           Soporte técnico           Sí
SOP-004  Soporte técnico           Soporte técnico           Sí
DEV-005  Desarrollo de software    Soporte técnico           No

Reporte de clasificación:
                      precision    recall  f1-score   support

Desarrollo de software       1.00      0.33      0.50         3
  Gestión de proyectos       0.67      1.00      0.80         2
Infraestructura y nube       1.00      0.50      0.67         2
    Políticas internas       1.00      1.00      1.00         2
       Soporte técnico       0.50      1.00      0.67         2

              accuracy                           0.73        11
             macro avg       0.83      0.77      0.73        11
          weighted avg       0.85      0.73      0.71        11
```

Figura 4. Categoría real contra la predicha en los 11 documentos de prueba.

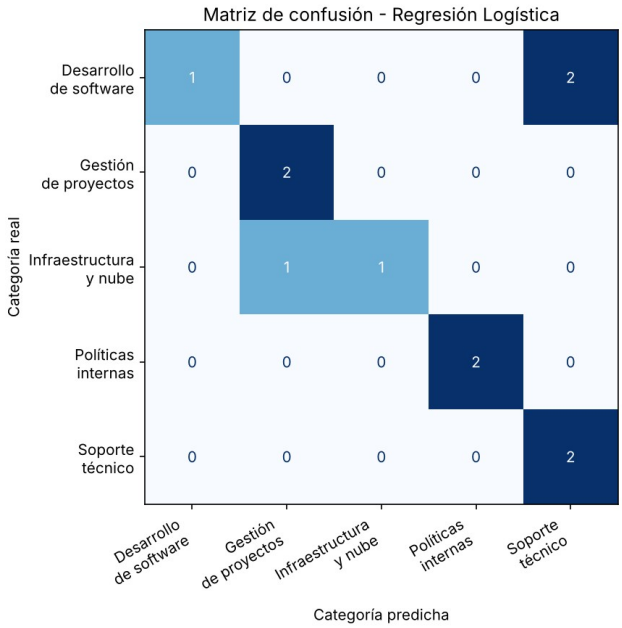


Figura 5. Matriz de confusión del modelo elegido.

Se equivocó en 3 de 11. DEV-005 y DEV-009 los mandó a Soporte técnico porque hablan de errores y correcciones, palabras que se repiten mucho en los tickets. INF-008 habla de costos y del líder de proyecto, por eso lo confundió con Gestión de proyectos. Con más documentos de entrenamiento estos errores deberían bajar.

5.3 Prueba de clasificación con documentos nuevos

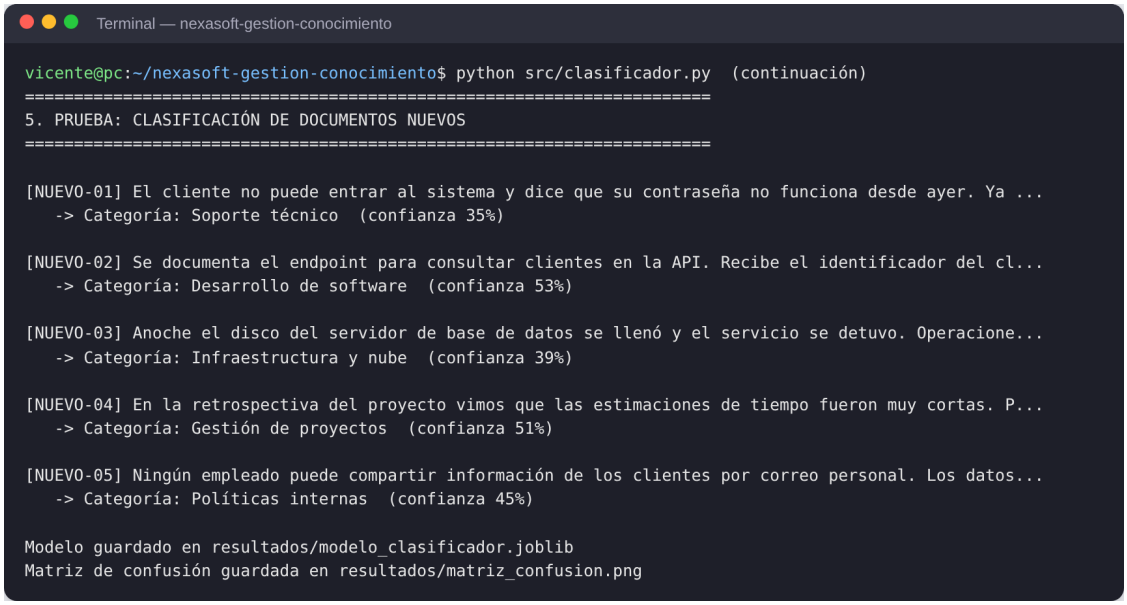


Figura 6. Clasificación de 5 documentos que el modelo nunca vio.

Documento nuevo	Categoría asignada	Confianza	¿Es correcta?
NUEVO-01: el cliente no puede entrar, cuenta bloqueada	Soporte técnico	35%	Sí
NUEVO-02: se documenta el endpoint de clientes en la API	Desarrollo de software	53%	Sí
NUEVO-03: el disco del servidor se llenó y se detuvo	Infraestructura y nube	39%	Sí
NUEVO-04: en la retrospectiva las estimaciones fueron cortas	Gestión de proyectos	51%	Sí
NUEVO-05: no compartir información de clientes por correo	Políticas internas	45%	Sí

Los 5 quedaron en la categoría correcta. La confianza sale entre 35% y 53% porque hay 5 categorías y pocos documentos, pero en todos los casos la categoría correcta fue la más alta.

5.4 Búsqueda semántica: consultas de ejemplo

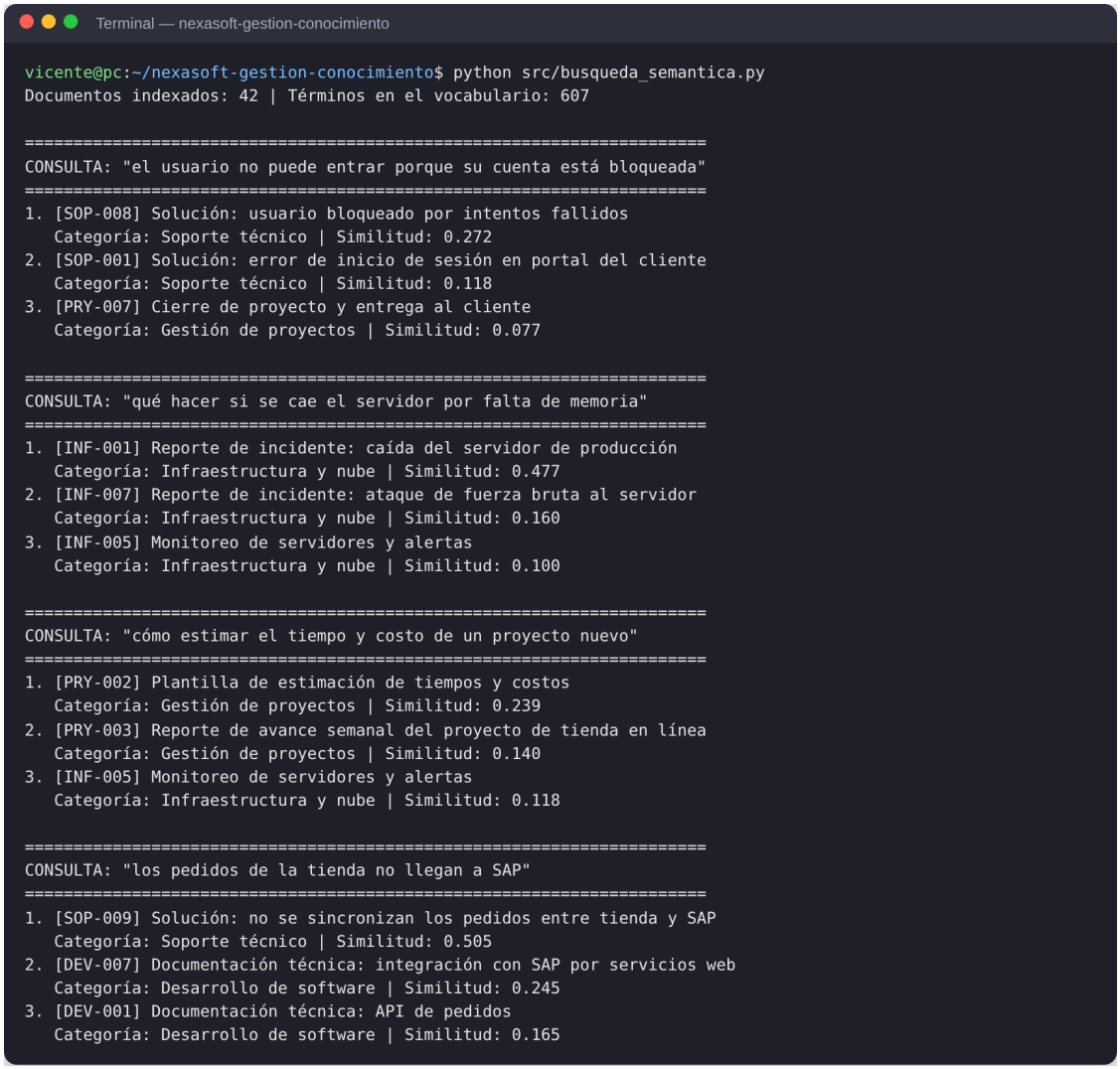


Figura 7. Ejecución de busqueda_semantica.py con cuatro consultas.

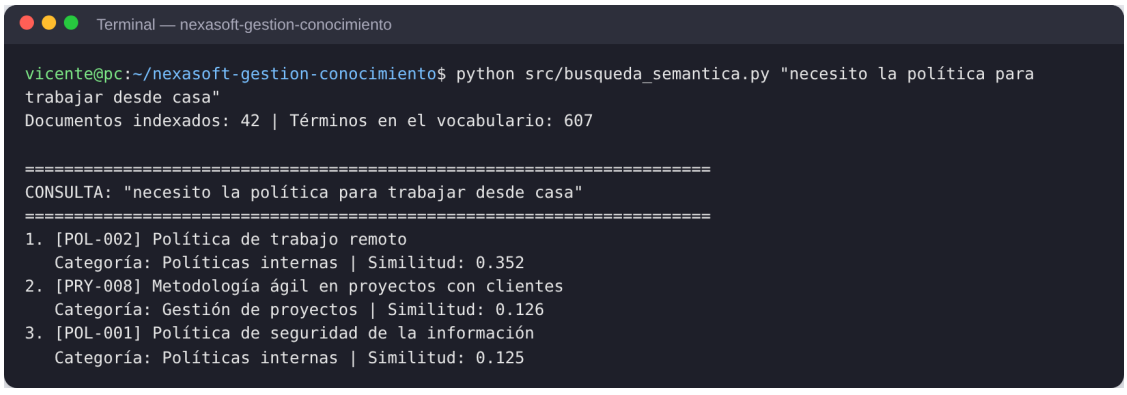


Figura 8. Búsqueda con una consulta escrita al momento.

Consulta	Documentos encontrados (similitud)	¿Por qué responde?
el usuario no puede entrar porque su cuenta está bloqueada	1. SOP-008 Usuario bloqueado (0.272) 2. SOP-001 Error de inicio de sesión (0.118) 3. PRY-007 Cierre de proyecto (0.077)	Los dos primeros son soluciones de Soporte sobre acceso y bloqueo, justo lo que se pregunta. El tercero tiene un puntaje muy bajo y solo comparte palabras como "usuario".
qué hacer si se cae el servidor por falta de memoria	1. INF-001 Caída del servidor (0.477) 2. INF-007 Ataque al servidor (0.160) 3. INF-005 Monitoreo y alertas (0.100)	El primero es exactamente el incidente de falta de memoria. Los tres son de Infraestructura y hablan del servidor.
cómo estimar el tiempo y costo de un proyecto nuevo	1. PRY-002 Plantilla de estimación (0.239) 2. PRY-003 Reporte de avance (0.140) 3. INF-005 Monitoreo (0.118)	La plantilla de estimación es la respuesta directa. El tercero sale por la palabra "tiempo", y por eso su puntaje es más bajo.
los pedidos de la tienda no llegan a SAP	1. SOP-009 Pedidos no se sincronizan con SAP (0.505) 2. DEV-007 Integración con SAP (0.245) 3. DEV-001 API de pedidos (0.165)	Encuentra la solución del ticket y además la documentación técnica de la integración. Esto muestra la relación Ticket → Sistema de la ontología.
necesito la política para trabajar desde casa	1. POL-002 Trabajo remoto (0.352) 2. PRY-008 Metodología ágil (0.126) 3. POL-001 Seguridad (0.125)	Encuentra la política de trabajo remoto aunque la consulta dice "desde casa" y el título dice "remoto", porque el texto del documento sí dice "trabajar desde casa".

En todas las consultas el primer resultado es el documento que de verdad responde la pregunta, y tiene un puntaje claramente más alto que los demás.

6. Evidencia de ejercicios de los Temas 1 a 6

Estas son las actividades del curso que me sirvieron para construir el proyecto:

Tema / actividad	Qué hice	Cómo ayudó al proyecto
Actividad 1: Ciclo de vida del conocimiento e IA	Analicé el ciclo de vida del conocimiento en Neoris, clasifiqué conocimiento tácito y explícito, y propuse dónde puede ayudar la IA.	Fue la base del mapa del ciclo de vida. Ahí ya proponía clasificar tickets y buscar soluciones parecidas con IA, que es lo que hace el prototipo.
Actividad 2: Ontología en Protégé	Hice una ontología de Neoris en Protégé con clases, propiedades e instancias.	Me ayudó a definir los conceptos y relaciones de la ontología de NexaSoft.
Ejercicios de clasificación (Temas 4 y 5)	[Describe aquí tus ejercicios de clasificación con TF-IDF o Naive Bayes.]	Base del clasificador.
Ejercicios de recuperación de información (Tema 6)	[Describe aquí tus ejercicios de búsqueda o similitud coseno.]	Base de la búsqueda semántica.

Actividad 1: diagrama del ciclo de vida

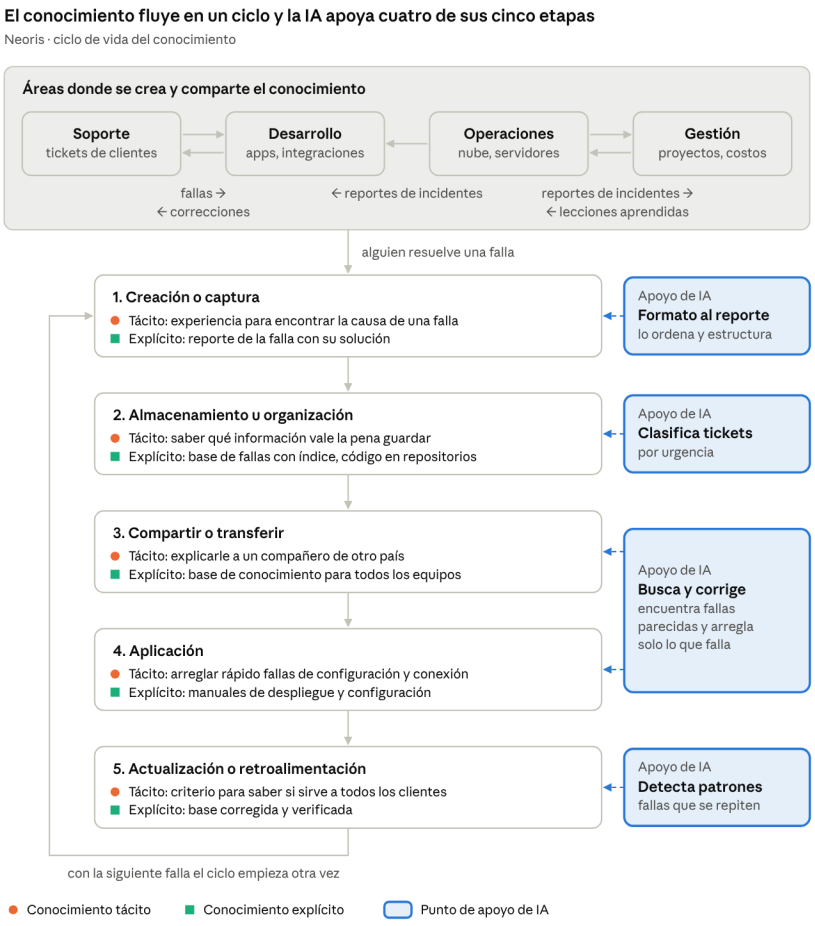


Figura 9. Diagrama que entregué en la Actividad 1, con los puntos donde la IA apoya el ciclo.

Actividad 2: ontología en Protégé

[Pega aquí la captura de tu ontología en Protégé]

Otros ejercicios

[Pega aquí capturas de tus ejercicios de clasificación o búsqueda de los temas 1 a 6]

7. Repositorio del proyecto

El código, los documentos de prueba y los resultados están en: github.com/vajalidk/nexasoft-gestion-conocimiento

Archivo o carpeta	Contenido
src/clasificador.py	Script de clasificación automática
src/busqueda_semantica.py	Script de búsqueda semántica
src/utilidades.py	Carga de documentos y palabras vacías
datos/documentos/	42 documentos de prueba en 5 carpetas
datos/nuevos/	5 documentos para probar el clasificador
resultados/	Salidas de ejecución, matriz de confusión y modelo
README.md y requirements.txt	Instrucciones para instalar y ejecutar

Para ejecutarlo: `pip install -r requirements.txt` y después `python src/clasificador.py` o `python src/busqueda_semantica.py "tu pregunta"`

8. Conclusión y siguientes pasos

Con este avance ya tengo las dos partes principales del sistema funcionando: el clasificador acomoda los documentos solos y la búsqueda encuentra el documento correcto con una pregunta normal. Lo que más me sirvió fue conectar el mapa y la ontología con el código, porque las categorías y las relaciones salen de ahí.

Para la entrega final quiero agregar más documentos para mejorar la exactitud, hacer una interfaz sencilla para buscar y probar embeddings para que la búsqueda entienda sinónimos.